AWS Resource Governance Dashboard (Technical Documentation and Authentication Guidance)

1. Project Overview

The AWS Resource Governance Dashboard (“AWS Dash”) is an internal engineering tool that gives organizations a unified, searchable, and manageable view of their AWS resources across multiple AWS accounts. It is designed to onboard new engineers quickly and to serve as the canonical reference for how the system is built, used, secured, and operated.

1.1 Architecture Breakdown

- Frontend: a Next.js React application providing the UI for searching, filtering, and tag editing.
- Backend: a FastAPI (Python) service that handles live communication with AWS via boto3.
- Authentication: OpenID Connect (OIDC) Single Sign-On (SSO). End users log in through the organization’s Identity Provider (IdP) — such as Okta or Azure AD — without ever needing AWS credentials.
- Authorization: Role-Based Access Control (RBAC). The backend validates OIDC JWT tokens and maps IdP groups to internal roles (viewer, tag_editor, admin), scoping access at both the action level and the AWS Account level (multi-tenancy).

2. Architecture Overview: Authentication & Authorization

AWS Dash uses an OpenID Connect (OIDC) Single Sign-On (SSO) architecture. Rather than managing user credentials itself, the system delegates login and identity entirely to an external Identity Provider (IdP) — Okta, Azure AD, Auth0, Keycloak, or any OIDC-compliant service. Authentication and authorization occur in two distinct phases.

2.1 Frontend Login (OIDC Code Flow)

An unauthenticated user hits the Next.js frontend and is immediately redirected to the configured IdP’s login page. After successful authentication, the IdP redirects back to /api/auth/callback with an authorization code. The frontend exchanges this code for an access_token and id_token, extracts user details (subject, email, groups, roles, tenant) from the JWT claims, and stores everything in an encrypted, HttpOnly browser cookie.

2.2 Backend Verification (Bearer Tokens)

Every API request from the frontend carries the access_token or id_token in an Authorization: Bearer <token> header. The FastAPI backend validates the JWT’s signature against the IdP’s JSON Web Key Set (JWKS) — checking issuer, audience, and expiration natively. It then maps JWT group claims to internal roles and enforces both multi-tenant isolation and AWS account scoping before any response is returned.

3. What Users Experience

3.1 Login

Users never create a username or password inside AWS Dash. They navigate to the dashboard URL, are redirected to their company's standard SSO login page (Okta, Microsoft, etc.), authenticate there, and land back in the dashboard already logged in. The redirect is seamless and the entire credential flow is invisible to the user.

3.2 Roles and Permissions

A user's capabilities are determined by the groups they belong to in their IdP directory. The backend maps those OIDC groups to one of three internal roles:

Role	Permissions	Description
Viewer	resources:read	Read-only access to all resources.
Tag Editor	resources:refresh, tags:update	View resources, refresh caches, and update AWS tags.
Admin	admin:manage	All permissions, plus administrative actions.

3.3 Tenants and AWS Account Restrictions

Tenants: Users belong to a tenant. In organizations managing multiple isolated environments, a user sees only data associated with their assigned tenant.

Account Restrictions: If a user is restricted to specific AWS accounts via IdP claims, they can only view and manage resources in those accounts.

4. Core Use Cases

AWS Dash addresses several common enterprise AWS management problems:

1. Centralized Inventory — Users can view resources such as S3 buckets, EC2 instances, RDS databases, CloudFront distributions, and Load Balancers across dozens of AWS accounts from a single pane of glass.
2. Secure Tagging — Tag Editors can update resource tags directly from the UI without needing IAM access to the AWS Console, ensuring consistent cost-allocation and governance tagging.
3. Multi-Tenant Isolation — The dashboard can be configured so that specific teams (tenants) only see AWS accounts that belong to them.
4. No Direct AWS Access for Users — Users never touch AWS Access Keys. The backend service assumes specific, least-privilege IAM roles in target AWS accounts on behalf of the user, based on their IdP group memberships.

5. Backend API Documentation

The FastAPI backend exposes several RESTful endpoints consumed by the frontend. All endpoints require a valid OIDC bearer token in the Authorization header.

5.1 GET /resources

Fetches a cached list of AWS resources, filtered by the provided query parameters.

Permission Required: resources:read

Query Parameters

Parameter	Description
search (string)	Keyword search across resource IDs, names, types, regions, or tags.
resource_type (string)	Filter by type (e.g., “S3 Bucket”, “EC2 Instance”).
region (string)	Filter by AWS region.
tag_key (string)	Return only resources possessing this tag key.
tag_value (string)	Return only resources where tag_key equals this value.
untagged_only (boolean)	If true, returns only resources with zero tags.
force_refresh (boolean)	If true, triggers a live background scan of all AWS accounts to update the cache. Requires resources:refresh permission.

Returns: Array of Resource objects.

5.2 POST /tags/update

Updates tags for a single specific resource.

Permission Required: tags:update

Request Body (JSON)

```
{
  "resource_id": "i-1234567890abcdef0",
  "resource_type": "EC2 Instance",
  "tags": {
    "Environment": "Production",
    "Owner": "Team-A"
  },
  "account_id": "123456789012"
}
```

Returns: {"status": "ok", "resource": {...}}
Notes: The backend verifies that the user’s tenant scope includes account_id before assuming the tagging role. All tagging operations are written to the audit log.

6. AWS IAM Policies Required

To operate securely, the backend should be deployed to a compute environment (such as ECS, EKS, or EC2) with an attached IAM Execution Role. The backend uses the AWS STS AssumeRole API to jump into target AWS accounts (member accounts) to read data or write tags.

Important: The backend should never be configured with long-lived static access keys in production. Always use IAM roles.

6.1 The Backend Execution Role (Base Role)

The IAM role attached to the backend container or server itself needs permission to assume roles in the member accounts.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "sts:AssumeRole",
      "Resource": [
        "arn:aws:iam::*:role/AwsDashReadOnlyRole",
        "arn:aws:iam::*:role/AwsDashTagEditorRole"
      ]
    }
  ]
}
```

6.2 The Member Account Read Role (AwsDashReadOnlyRole)

This role must be deployed into every AWS account that the dashboard scans. It allows the backend to inventory resources and read tags, and must trust the Backend Execution Role.

Permissions Needed

- AWS Organizations: organizations:ListAccounts, organizations:ListParents, organizations:DescribeOrganizationalUnit (only needed if scanning from the Org Management account).
- S3: s3:ListAllMyBuckets, s3:GetBucketTagging
- EC2: ec2:DescribeInstances
- RDS: rds:DescribeDBInstances, rds:ListTagsForResource
- ELBv2: elasticloadbalancing:DescribeLoadBalancers, elasticloadbalancing:DescribeTags
- CloudFront: cloudfront:ListDistributions, cloudfront:ListTagsForResource
- Resource Groups Tagging API: tag:GetResources

6.3 The Member Account Write Role (AwsDashTagEditorRole)

This role must be deployed into every AWS account where users are allowed to edit tags via the dashboard. It must trust the Backend Execution Role.

Permissions Needed

- S3: s3:PutBucketTagging
- EC2: ec2:CreateTags

- RDS: rds:AddTagsToResource
- ELBv2: elasticloadbalancing:AddTags
- CloudFront: cloudfront:TagResource

7. Environment Variable Reference

The system is configured via the .env file. Every variable listed below is required for secure operation.

7.1 Frontend OIDC Setup

These variables tell the Next.js frontend how to negotiate with the OIDC provider.

Variable	Description	Example
APP_BASE_URL	The public URL of the frontend. Used for OIDC redirects.	http://localhost:3000
OIDC_CLIENT_ID	The Client ID issued by your IdP for the AWS Dash application.	0oa...
OIDC_CLIENT_SECRET	The Client Secret issued by your IdP.	...
OIDC_AUTHORIZATION_URL	The IdP's authorization endpoint.	https://your-idp.example.com/.../authorize
OIDC_TOKEN_URL	The IdP's token endpoint.	https://your-idp.example.com/.../token
OIDC_REDIRECT_URI	The frontend callback URL the IdP redirects to after login.	http://localhost:3000/api/auth/callback
OIDC_SCOPES	Scopes requested from the IdP. Must include openid.	openid profile email groups
AUTH_COOKIE_SECRET	A random string (≥ 32 chars) used to AES-256 encrypt the session cookie.	super-secret-random-string...
AUTH_BYPASS_LOGIN	Set to false to enable a dummy local-dev session for testing without SSO.	false

7.2 Backend JWT Verification

These variables tell the FastAPI backend how to cryptographically verify the tokens it receives.

Variable	Description	Example
OIDC_ISSUER	The expected issuer (iss) of the JWT.	https://your-idp.example.com/oauth2/default
OIDC_AUDIENCE	The expected audience (aud) of the JWT.	api://aws-dash
OIDC_JWKS_URL	URL for the IdP's public keys. Defaults to <issuer>/.well-known/jwks.json.	https://.../v1/keys

7.3 Claim Mapping and Role Assignment

These variables tell the backend where to find user details inside the JWT, and how to interpret them.

Variable	Description	Example
AUTH_GROUPS_CLAIM	JWT claim that holds the user's groups.	groups
AUTH_ROLES_CLAIM	JWT claim that holds the user's roles.	roles
AUTH_TENANT_CLAIM	JWT claim that specifies the user's tenant.	aws_dash_tenant
AUTH_ACCOUNTS_CLAIM	JWT claim that specifies allowed AWS accounts.	aws_accounts

The backend matches the string array found in AUTH_GROUPS_CLAIM against three comma-separated lists. A match grants the corresponding internal role:

Variable	Example Value
AUTH_VIEWER_GROUPS	aws-dash-viewers
AUTH_TAG_EDITOR_GROUPS	aws-dash-tag-editors
AUTH_ADMIN_GROUPS	aws-dash-admins

7.4 Tenancy and Account Scoping

Variable	Description	Example
AUTH_DEFAULT_TENANT	The name of the default tenant.	default
AUTH_ALLOW_DEFAULT_TENANT	If true, users missing a tenant claim fall back to the default tenant. If false, they are denied.	false
AUTH_TENANT_ACCOUNTS_JSON	JSON mapping of Tenant IDs to arrays of AWS Account IDs, controlling which accounts each tenant may access.	{"default":["123456789012"]}

7.5 Testing and Development Overrides

AUTH_ENABLED — if set to false, the backend skips JWT verification entirely and treats every request as a default local admin. Never disable this in production.

AUTH_TEST_MODE — if true, the backend accepts unverified base64-encoded JSON instead of a signed JWT. For automated testing only.

Provide static AWS Keys

AWS_ACCESS_KEY_ID=your-access-key-here

AWS_SECRET_ACCESS_KEY=your-secret-key-here

AWS_SESSION_TOKEN=your-session-token-here # Optional

Security Notice: Always set AUTH_ENABLED=true and AUTH_TEST_MODE=false in production. If AUTH_COOKIE_SECRET is ever compromised, rotate it immediately — this forcibly logs out all active sessions.

8. Development Workflow

For a new developer joining the codebase:

5. Frontend — Located in /frontend. It is a Next.js application. Use npm run dev to start it locally. Check /frontend/lib/server-auth.ts for how OIDC cookies are parsed.
6. Backend — Located in /backend. It is a FastAPI application. Use uvicorn app.main:app --reload to start it locally.
7. AWS Integration — The core AWS logic is in backend/app/services/inventory.py. To add support for a new AWS resource type (e.g., DynamoDB):
   - Add a _list_dynamodb_resources function.
   - Call it inside collect_live_resources.
   - Update update_resource_tags to handle the resource_type_lower == "dynamodb table" condition for write operations.
   - Add the necessary dynamodb:DescribeTable and dynamodb:TagResource permissions to the IAM policies.